Métodos Formais

Prof. Júlio Machado

Exercícios

Programação e Verificação com Dafny (Sets, Multisets)

1) Acesse o tutorial do Dafny. Execute as atividades indicadas em nas seguintes seções:

- Collection Types / Sets / Multisets / Maps
  https://dafny.org/dafny/OnlineTutorial/ValueTypes
- Sets https://dafny.org/dafny/OnlineTutorial/Sets

2) Explique com suas palavras o que cada asserção descrita no seguinte código significa:

```
method FazAlgo(a: array<int>, n: int, e: int)
  requires 0 <= n < a.Length
  modifies a
  ensures multiset(a[..n+1]) == multiset(a[..n]) + multiset{e}
{
  a[n] := e;
  assert a[..n+1] == a[..n] + [e];
}
```

3) O uso de sequências, conjuntos e multiconjuntos é muito útil para simplificar especificações sobre outros tipos de coleções. Deseja-se especificar um predicado capaz de indicar se um determinado array é uma permutação de outro array. Tal predicado é bastante útil na especificação de algoritmos de ordenação, por exemplo.

a) De que forma você verificaria se um array é uma permutação de outro? Pense em um algoritmo antes de executar as instruções do exercício que segue.

b) Seja a seguinte definição: um array *a* é uma permutação de um array *b* se e somente se para todo elemento *e*, tem-se que a cardinalidade de *e* em *a* é a mesma que a cardinalidade de *e* em *b*. Com base nessa definição, defina um predicado $permutacao: sequencia\langle\mathbb{Z}\rangle \times sequencia\langle\mathbb{Z}\rangle \rightarrow \mathbb{B}$, e o utilize para verificar se dois arrays são uma permutação entre si. Dica: faça uso de multiconjuntos como um elemento auxiliar para especificar o predicado.

```
ghost predicate Permutacao(a:seq<int>, b:seq<int>)
{
}
```

```
method Main()
{
    var a := new nat[3];
    a[0], a[1], a[2] := 1, 2, 3;
    var b := new nat[3];
    b[0], b[1], b[2] := 3, 1, 2;
```

```
    assert a[..] == [1,2,3] && b[..] == [3,1,2]; //asserção necessária
para o verificador conhecer o conteúdo do array
    assert Permutacao(a[..], b[..]);
}
```

4) Especifique, implemente e prove a correção de um método que recebe um array de números inteiros, e dois índices *i* e *j* e faz uma troca (*swap*) entre os elementos encontrados nas posições *i* e *j*. Não esqueça de realizar um teste unitário em um método *Main* para verificar se sua especificação é adequada. Utilize a seguinte assinatura para o método:

```
method TrocaElementos(a: array<int>, i: int, j: int)
```

5) Deseja-se implementar e verificar a correção do método de ordenação de arrays conhecido como BubbleSort. Os seguintes predicados já estão definidos e podem ser utilizados na verificação do método *BubbleSort* a ser implementado em Dafny. A fim de simplificar, estamos utilizando somente arrays de inteiros. Não esqueça de realizar um teste unitário em um método *Main* para verificar se sua especificação é adequada.

```
ghost predicate Permutacao(a:seq<int>, b:seq<int>)
{
    multiset(a) == multiset(b)
}

ghost predicate OrdenadoEntre(a:array<int>, e:int, d:int)
  requires 0 <= e <= d <= a.Length
  reads a
{
    forall i,j ::e <= i <= j < d ==> a[i] <= a[j]
}

ghost predicate Ordenado(a:array<int>)
  reads a
{
    OrdenadoEntre(a,0,a.Length)
}
```

```
method BubbleSort(a:array<int>)
  ensures Ordenado(a)
  ensures Permutacao(a[..], old(a[..]))
  modifies a
```